

PHP in kleinen Schritten

16 Mini-Aufgaben • ein Formular • jeweils eine kleine Änderung

Das Formular und das Styling sind fertig. Du ergänzt pro Aufgabe nur den markierten PHP-Bereich. Arbeite eine Aufgabe nach der anderen ab und prüfe immer die angegebenen Testfälle.

Heute nicht: Sessions, Cookies, Datenbank, Klassen, verschachtelte Arrays oder eine große Mehrseiten-Anwendung. Hier geht es nur um Eingaben, einfache Arrays und kleine Kontrollstrukturen.

So startest du

1. Entpacke das Starterpaket vollständig. Starte PHP-START.cmd im Ordner php_kleine_schritte.
2. Öffne <http://127.0.0.1:8085/> im Browser. Das Serverfenster bleibt offen.
3. Wähle Aufgabe 01. Öffne daneben deren auswertung.php in VS Code.
4. Ergänze nur zwischen „AB HIER ARBEITEN“ und „BIS HIER ARBEITEN“. Speichere und sende das Formular erneut.

Pro Aufgabe dieselben drei Dateien

Datei	Was du damit machst
formular.html	Im Browser ausfüllen und absenden. Bis Aufgabe 15 nicht umbauen.
style.css	Ist fertig. Muss nicht bearbeitet werden.
auswertung.php	Hier ergänzt du den kleinen PHP-Arbeitsauftrag.

Wichtig: Die Aufgabenordner sind unabhängig. Benötigte Vorarbeiten stehen bereits in der jeweiligen PHP-Datei. Du musst keine gelöste Voraufgabe kopieren.

XAMPP-Alternative: Ordner nach C:\xampp\htdocs kopieren, Apache starten und http://localhost/php_kleine_schritte/ öffnen. Keine Datenbank nötig. Nicht über file:// oder per Doppelklick absenden.

Das Formular bleibt gleich

Vier Felder reichen. Die Standardwerte sind Mia, Paris und 2 Personen. Der Newsletter ist zunächst nicht angehakt.

Sichtbares Feld	name / Schlüssel	Übertragener Wert
Vorname	vorname	Mia oder dein Text; auch leer testbar
Reiseziel · Dropdown	ziel	berlin, paris, rom oder leer
Personen · Dropdown	personen	1, 2, 3, 4 oder leer; zunächst Text
Newsletter · Checkbox	newsletter	ja; ohne Haken fehlt dieser Schlüssel

Die Felder haben absichtlich kein `required`. Ein leerer Wert soll PHP erreichen können. **Leer ist nicht dasselbe wie fehlend.** Ein direkter Aufruf von `auswertung.php` sendet keine POST-Formulardaten.

GET und POST sind sichtbar vorbereitet

```
<form action="auswertung.php" method="post">
<form action="auswertung.php" method="get">
```

Diese zwei Zeilen sind Alternativen, nicht zwei Formulare in einer Datei. Die passende Zeile ist pro Aufgabe schon eingestellt. Nur in Aufgabe 16 wechselst du sie selbst.

Du arbeitest nur mit \$ausgabe

```
$ausgabe = "";

// Hier kommt deine kleine PHP-Auswertung hin.
```

Der vorbereitete HTML-Bereich zeigt `$ausgabe` als Text an. Mit `"\n"` erzeugst du neue Zeilen; mit `.=` hängst du Text an. Du brauchst keine HTML-Tags in die Variable zu schreiben.

Die Ausgabe mit `htmlspecialchars` ist bereits fertig. Sonderzeichen werden als Text angezeigt. Auch kleine Typprüfungen und das Einlesen von Werten sind ab Aufgabe 05 teilweise vorgegeben; der Arbeitsauftrag nennt dir die verfügbaren Variablen.

Grundlage: Kapitel 6, S. 17–21; Kapitel 4, HTML Teil 2, S. 23–30. Neue, bewusst vereinfachte Aufgaben; technische Hinweise mit PHP-Handbuch abgeglichen. Quellen auf Seite 11.

01

POST • 5–10 Min.

POST-Daten ansehen

aufgabe_01/auswertung.php

Erkennen, was das Formular an PHP übergibt.

Bereitgestellt: Nur das Ausgabe-Gerüst ist vorbereitet.

1. Öffne `aufgabe_01/formular.html`. Sende die voreingestellten Werte ab.
2. Zeige in `$ausgabe` den gesamten Inhalt von `$_POST` an. Nutze `print_r` mit `true`.
3. Sende noch einmal mit angehaktem Newsletter. Vergleiche beide Ausgaben.

Hinweis: `print_r($array, true)` liefert die Arraydarstellung als Text. Diesen Text weist du `$ausgabe` zu.

Test	Erwartung
Mia, Paris, 2; ohne Haken	vorname, ziel und personen sind enthalten.
Dieselben Werte; mit Haken	Zusätzlich erscheint newsletter mit dem Wert ja.

02

GET • 5–10 Min.

Dasselbe Formular mit GET

aufgabe_02/auswertung.php

GET und POST bei gleichen Feldnamen vergleichen.

Bereitgestellt: Nur das Ausgabe-Gerüst ist vorbereitet.

1. Nutze `aufgabe_02/formular.html`. Hier ist `method="get"` bereits eingestellt.
2. Zeige jetzt `$_GET` statt `$_POST` im Ausgabefeld an.
3. Sende Mia, Paris und 2. Vergleiche die Adresszeile mit Aufgabe 01.

Hinweis: Das Formular bleibt gleich. Geändert sind die Übertragungsmethode und das in PHP gelesene Array.

Test	Erwartung
Mia, Paris, 2	Dieselben drei Feldnamen wie bei POST.
Adresszeile prüfen	Die GET-Feldwerte stehen hinter dem Fragezeichen.

03

POST · 5–10 Min.

Ist der Schlüssel vorhanden?

aufgabe_03/auswertung.php

Vorhandensein prüfen, bevor ein Wert benutzt wird.

Bereitgestellt: Nur das Ausgabe-Gerüst ist vorbereitet.

1. Prüfe mit `isset`, ob `$_POST["vorname"]` gesetzt und nicht null ist.
2. Setze `$ausgabe` auf "Vorname vorhanden." oder "Vorname fehlt."
3. Prüfe auch einen leeren Namen. Noch nicht auf leeren Text prüfen.

Hinweis: Ein leerer Text `""` ist für `isset` trotzdem vorhanden. Rufe `auswertung.php` direkt auf, um ohne POST-Daten zu testen.

Test	Erwartung
Mia oder ein leerer Name senden	Vorname vorhanden.
auswertung.php direkt öffnen	Vorname fehlt.

04

GET · 5–10 Min.

Fehlend, leer oder ausgefüllt?

aufgabe_04/auswertung.php

Drei Eingabesituationen unterscheiden.

Bereitgestellt: Ein kleiner Typcheck ist fertig. Die Prüfung auf fehlend und leer ergänzt du.

1. Prüfe zuerst mit `isset`, ob `vorname` in `$_GET` vorhanden ist.
2. Prüfe danach, ob der Name nach `trim` leer ist. Meldung: "Bitte Vorname eingeben."
3. Bei gefülltem Namen soll "Hallo Mia." erscheinen. Bei fehlendem Schlüssel: "Vorname fehlt."

Hinweis: Der Typcheck für unerwartete Array-Eingaben ist fertig. Ergänze nur den markierten `else`-Block.

Test	Erwartung
Ohne Query / <code>?vorname=</code>	Vorname fehlt. / Bitte Vorname eingeben.
Mia oder nur Leerzeichen	Hallo Mia. / Bitte Vorname eingeben.

05

POST • 5–10 Min.

Ein eigenes assoziatives Array

aufgabe_05/auswertung.php

Werte unter sprechenden Schlüsseln sammeln.

Bereitgestellt: \$vorname, \$ziel und \$personen sind als Texte vorbereitet.

1. Lege \$reise mit den Schlüsseln vorname, ziel und personen an.
2. Übernimm die vorbereiteten Variablen als Werte. Schreibe nicht Mia oder Paris fest in den Code.
3. Lies ausschließlich \$reise["ziel"] aus und weise den Wert \$ausgabe zu.

Hinweis: Schreibweise: "schluessel" => \$wert. Die Formularauswahl Paris überträgt den Wert paris.

Test	Erwartung
Paris auswählen	paris
Berlin auswählen	berlin

06

GET • 5–10 Min.

Dropdown-Wert nachschlagen

aufgabe_06/auswertung.php

Einen Schlüssel zum Nachschlagen verwenden.

Bereitgestellt: \$ziel sowie der Katalog \$ziele mit Berlin, Paris und Rom sind fertig.

1. Nutze \$ziel als Schlüssel im vorbereiteten Array \$ziele.
2. Prüfe vor dem Zugriff mit isset, ob dieser Schlüssel im Katalog vorhanden ist.
3. Gib den Städtenamen aus. Andernfalls: "Unbekanntes Ziel."

Hinweis: Zuerst prüfen, dann lesen: \$ziele[\$ziel]. Teste den unbekannten Wert über ?ziel=wien in der URL.

Test	Erwartung
Paris auswählen	Paris
?ziel=wien oder ?ziel=	Unbekanntes Ziel.

07

POST · 5–10 Min.

Das Array mit foreach durchlaufen

aufgabe_07/auswertung.php

Schlüssel und Wert jedes Eintrags ausgeben.

Bereitgestellt: \$reise enthält drei vorbereitete Schlüssel-Wert-Paare. \$ausgabe ist anfangs leer.

1. Durchlaufe das vorbereitete Array \$reise mit foreach.
2. Hänge pro Eintrag "Schlüssel: Wert" an \$ausgabe an.
3. Füge nach jedem Eintrag einen Zeilenumbruch mit "\n" ein.

Hinweis: foreach (\$array as \$schluessel => \$wert) liefert beides. Mit .= hängst du Text an die vorhandene Ausgabe an.

Test	Erwartung
Mia, Paris, 2	Drei Zeilen: vorname: Mia / ziel: paris / personen: 2
Personen auf 4 ändern	Die letzte Zeile lautet personen: 4.

08

GET · 10 Min.

In der Schleife gezielt suchen

aufgabe_08/auswertung.php

foreach mit einer Bedingung verbinden.

Bereitgestellt: \$ziel und \$ziele sind vorbereitet.

1. Setze \$ausgabe zuerst auf "Unbekanntes Ziel.". Durchlaufe danach \$ziele.
2. Vergleiche jeden Schlüssel mit \$ziel. Bei Gleichheit übernimm den Städtenamen.
3. Beende die Suche nach dem Treffer mit break. Nutze diesmal keinen direkten Zugriff \$ziele[\$ziel].

Hinweis: Gedanke in Pseudocode: FÜR JEDEN Eintrag → WENN Schlüssel = Auswahl → Namen merken → Schleife beenden.

Test	Erwartung
Rom auswählen	Rom
?ziel=wien	Unbekanntes Ziel.

09

POST · 5–10 Min.

Eine Checkbox mit if/else prüfen

aufgabe_09/auswertung.php

Mit einem optionalen Feld arbeiten.

Bereitgestellt: Nur das Ausgabe-Gerüst ist vorbereitet.

1. Prüfe, ob newsletter in \$_POST vorhanden ist und den Wert "ja" hat.
2. Bei gesetztem Haken: "Newsletter gewünscht.". Sonst: "Kein Newsletter.".
3. Sende das Formular einmal mit und einmal ohne Haken.

Hinweis: Eine nicht markierte Checkbox wird hier nicht gesendet. Prüfe deshalb zuerst isset und erst dann den Wert.

Test	Erwartung
Newsletter angehakt	Newsletter gewünscht.
Newsletter nicht angehakt	Kein Newsletter.

10

GET · 5–10 Min.

Mit if / elseif einordnen

aufgabe_10/auswertung.php

Mehrere Fälle in einer festen Reihenfolge prüfen.

Bereitgestellt: \$personen ist bereits als ganze Zahl vorbereitet; 0 bedeutet fehlend oder ungültig.

1. Für \$personen === 0: "Bitte 1 bis 4 Personen auswählen.".
2. Für 1 Person: "Alleinreise.". Für 2 oder 3: "Kleine Gruppe.".
3. Für 4 Personen: "Vierergruppe.". Verwende if, elseif und else.

Hinweis: Die Vorlage liefert eine ganze Zahl von 1 bis 4. Fehlende und ungültige Werte werden vorher auf 0 gesetzt.

Test	Erwartung
1 / 2 / 3 / 4 Personen	Alleinreise. / Kleine Gruppe. / Kleine Gruppe. / Vierergruppe.
?personen=0 oder ?personen=abc	Bitte 1 bis 4 Personen auswählen.

11

POST • 10 Min.

Mit switch eine Begrüßung wählen

aufgabe_11/auswertung.php

Eine Auswahl auf feste Fälle abbilden.

Bereitgestellt: \$ziel ist als Text vorbereitet.

1. Verwende switch (\$ziel). Lege je einen case für berlin, paris und rom an.
2. Verwende die Texte "Guten Tag.", "Bonjour." und "Buongiorno.". Beende jeden case mit break.
3. Ergänze default mit "Unbekanntes Ziel."

Hinweis: case "wert": enthält die Anweisungen für diesen Wert. default behandelt eine nicht zugeordnete Auswahl.

Test	Erwartung
Berlin / Paris / Rom	Guten Tag. / Bonjour. / Buongiorno.
Im Dropdown "Bitte wählen"	Unbekanntes Ziel.

12

GET • 5–10 Min.

Mit for nummerieren

aufgabe_12/auswertung.php

So oft ausgeben, wie Personen gewählt wurden.

Bereitgestellt: \$personen und die Eingabepfung sind fertig. Ergänze nur die Schleife.

1. Ergänze im vorbereiteten else-Block eine for-Schleife.
2. Starte den Zähler bei 1 und zähle bis einschließlich \$personen.
3. Hänge pro Durchlauf "Person " und die Nummer mit einem Zeilenumbruch an \$ausgabe an.

Hinweis: for hat Startwert, Bedingung und Zähleränderung. Die Prüfung auf eine ungültige Personenanzahl ist bereits fertig.

Test	Erwartung
2 Personen	Person 1 und Person 2 auf getrennten Zeilen.
4 Personen	Genau vier Zeilen, letzte Zeile: Person 4.

13

POST • 5–10 Min.

Dieselbe Ausgabe mit while

aufgabe_13/auswertung.php

Zähler und Wiederholung selbst steuern.

Bereitgestellt: `$personen` und die Eingabeprüfung sind fertig. Ergänze nur die Schleife.

1. Erzeuge dieselbe Ausgabe wie in Aufgabe 12, aber mit `while`.
2. Lege den Zähler vor der Schleife mit 1 an. Prüfe `$i <= $personen`.
3. Erhöhe den Zähler im Schleifenrumpf nach jeder Ausgabe.

Hinweis: Ohne Zähleränderung kann die Schleife endlos laufen. Das vorbereitete Gerüst prüft die Personenanzahl.

Test	Erwartung
1 Person	Genau eine Zeile: Person 1.
3 Personen	Genau drei nummerierte Zeilen.

14

GET • 5–10 Min.

Mit do-while wiederholen

aufgabe_14/auswertung.php

Die Schleifenbedingung am Ende prüfen.

Bereitgestellt: `$personen` und die Prüfung vor der Schleife sind fertig.

1. Wiederhole die nummerierte Ausgabe mit `do-while`. Der Zähler startet bei 1.
2. Gib zuerst die Zeile aus, erhöhe den Zähler und prüfe danach die Bedingung.
3. Lass die vorhandene Prüfung auf `$personen == 0` vor der Schleife unverändert.

Hinweis: `do { ... } while (Bedingung);` endet mit einem Semikolon. Die äußere Prüfung verhindert eine Ausgabe bei ungültiger Anzahl.

Test	Erwartung
2 Personen	Genau zwei nummerierte Zeilen.
?personen=0	Nur die Meldung zur ungültigen Auswahl; keine Person 1.

15

POST • 10 Min.

Aus der Auswahl einen Preis berechnen

aufgabe_15/auswertung.php

Arrayzugriff und eine einfache Rechnung verbinden.

Bereitgestellt: \$ziel, \$personen und \$preise sind fertig. Alle Preise sind fiktive Übungswerte.

1. Prüfe: Gibt es \$ziel in \$preise und ist \$personen größer als 0?
2. Berechne nur dann: Preis pro Person mal Personenanzahl.
3. Gib z. B. "paris: 240,00 Euro" aus. Sonst: "Bitte gültiges Ziel und 1 bis 4 Personen auswählen."

Hinweis: Vorbereitete Preise pro Person: berlin 80, paris 120, rom 100. `number_format($betrag, 2, ",", ".")` formatiert die Zahl.

Test	Erwartung
Paris mit 2 Personen	paris: 240,00 Euro
Berlin mit 3 / kein Ziel	berlin: 240,00 Euro / Hinweis auf gültige Auswahl

16

POST → GET • 10–15 Min.

Eine Auswertung für GET und POST

aufgabe_16/auswertung.php

Die passende Datenquelle ausdrücklich wählen.

Bereitgestellt: \$methode und ein leeres \$daten sind vorbereitet.

1. Fülle \$daten abhängig von \$methode: bei POST aus \$_POST, bei GET aus \$_GET.
2. Lies den Vornamen nur aus \$daten. Ergänze fehlend/leer/gefüllt wie in Aufgabe 04; bei Erfolg z. B. "POST: Hallo Mia."
3. Teste zuerst POST. Ändere dann im selben Formular nur `method="post"` in `method="get"` und teste erneut.

Hinweis: Benutze kein \$_REQUEST. Ein leerer POST darf nicht still auf GET-Werte ausweichen. Beim direkten GET-Aufruf fehlt der Name.

Test	Erwartung
Mia einmal per POST, einmal per GET	POST: Hallo Mia. / GET: Hallo Mia.
auswertung.php ohne Formularwerte	Vorname fehlt.

Nur die wichtigsten Bausteine

Baustein	Bedeutung
<code>isset(\$daten["feld"])</code>	Schlüssel ist gesetzt und sein Wert ist nicht null. Ein leerer Text ergibt true.
<code>trim(\$text) === ""</code>	Text ist nach Entfernen äußerer Leerzeichen leer.
<code>\$wert = \$daten["feld"] ?? "";</code>	Fehlenden beziehungsweise null-Wert durch leeren Text ersetzen.
<code>is_string(\$wert)</code>	Prüft, ob der Wert Text ist. In mehreren Vorlagen schon eingebaut.
<code>\$ausgabe .= \$text . "\n";</code>	Text und einen Zeilenumbruch an die Ausgabe anhängen.
<code>break;</code>	Beendet hier switch oder die gerade betrachtete Schleife.

Stolperstelle: `empty` behandelt auch `"0"` als leer. Deshalb prüfen wir leere Texte in diesen Aufgaben ausdrücklich mit `trim($text) === ""`. Vorhandensein, Datentyp und erlaubter Wert sind unterschiedliche Prüfungen.

So prüfst du jede Aufgabe

- Einmal mit den angegebenen Standardwerten testen.
- Mindestens den zweiten Testfall vom Aufgabenblatt durchführen.
- Ausgabe mit der Erwartung vergleichen. Erst dann weitergehen.

GET: Query-Werte kommen in `$_GET` an. **POST:** Die üblichen Formular Daten im Request-Body kommen in `$_POST` an. POST allein ist keine Verschlüsselung. Nutze nur fiktive Daten.

Grundlagen und Quellen

Unterrichtsbezug: „PHP-Kurs – Kapitel 6: PHP DB“, S. 17–21; „Kapitel 4: HTML, Teil 2“, S. 23–30; die zuvor bereitgestellte PHP-Arrays-Webvariante. Diese Aufgabenserie, ihre vier Formularfelder und die konkreten Testfälle sind neu und gegenüber der Mehrseiten-Demo vereinfacht.

Formulardaten: www.php.net/manual/en/language.variables.external.php

Arrays: www.php.net/manual/de/language.types.array.php

isset / empty: www.php.net/manual/de/function.isset.php

foreach: www.php.net/manual/de/control-structures.foreach.php

switch: www.php.net/manual/de/control-structures.switch.php

HTML-Ausgabe: www.php.net/manual/de/function.htmlspecialchars.php